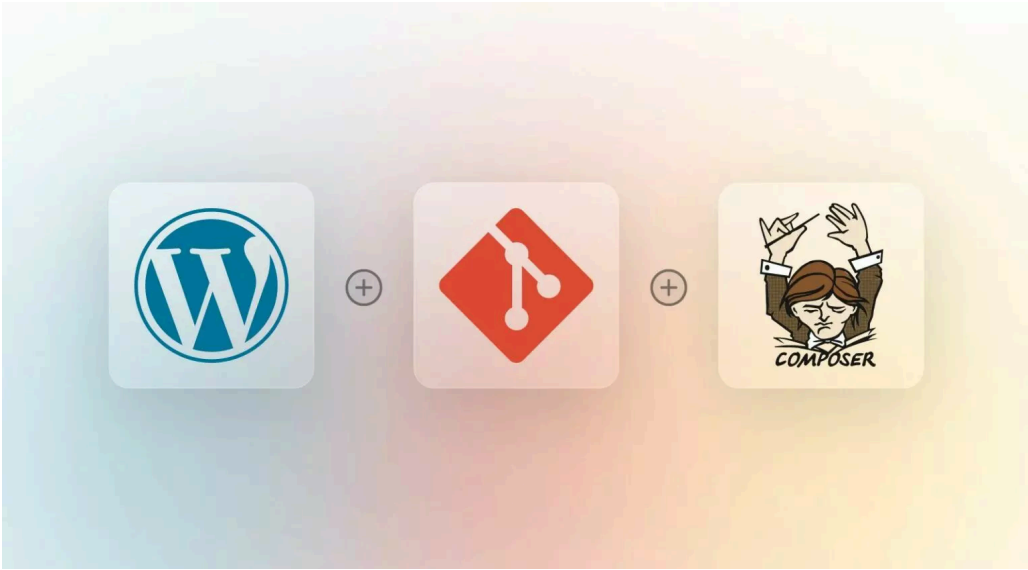


Managing Your WordPress Site With Git and Composer

#(https://deliciousbrains.com/storing-wordpress-in-git/) PUBLISHED JUL 6, 2022

 BY ROSS WINTLE, SOFTWARE DEVELOPER



Managing WordPress with Git and Composer has some big advantages. In particular, it can give you version control and helps get new people up to speed quickly. But it's not for every WordPress project. In this article we'll look at the pros and cons of this approach, and I'll show you how to manage WordPress core, themes, and plugins using Git for version control and Composer for dependency management. Finally, we'll look at moving WordPress into a subdirectory on Git for increased security.

Composer is a dependency manager, a tool that lets you say what software components (and which versions) your project depends on, and that automatically downloads and installs those dependencies.

Git is a version control system that keeps track of changes to the code of a project, enables engineers to collaborate on a codebase, and aids deployment of applications to servers. While it is possible to only version control parts of your WordPress project, such as a custom-built theme or your own custom plugins, this article focuses on using the powerful and reliable combination of Git and Composer to keep control of the whole web application that you are building with WordPress.

Table of Contents

- 1What is Composer?
- 2Why Git?
- 3Advantages of the Composer + Git Approach
- 4Installing Composer

1Configuring Composer

2Installing WordPress Core Using Composer
- 5Creating a Git Repository

1Adding Plugins and Themes From the WordPress Repository

2Adding Custom or Third-party Plugins and Themes

3Some Notes on Using Git in This Way

4Deployments and Updates
- 6Next Steps: Moving WordPress
- 7Next Steps: Using Git Submodules to Manage Themes and Plugins
- 8Next Steps: Restructuring WordPress
- 9Wrapping Up

What is Composer?

Composer is the de facto dependency manager for PHP. A dependency is a “package” of code that your project needs to work. Perhaps a library that handles API requests, or authentication. It's like a plugin for the PHP language.

In a WordPress context, these dependencies are:

- WordPress core – the main WordPress software that you normally download or install using a one-click software installer.
- Plugins and “must-use” plugins.
- Themes.
- Any other PHP packages you want to make use of.

The code of the specified dependencies makes up an application. You use the dependency manager to keep control of the application, so that you have a known and reproducible codebase at any given time.

The packages that your project requires, and their versions, are listed in a `composer.json` file, along with some other settings that tell Composer what to do.

The `composer.json` file doesn't have to contain exact version numbers of packages. You can tell it to use "1.2.x" or "the latest development version". Composer has an `update` command that figures out which versions of which packages need to be installed and saves the packages and versions in a `composer.lock` file.

The packages listed in the `composer.lock` file are what is actually installed by Composer, and I note that this file is always generated by Composer commands. You should not edit it manually.

Once you commit `composer.lock` to your project repo, anyone working on the project is locked into using those same versions, and can get up and running with a copy of the code/application just by running `composer install`. It's quick and ensures everyone is working with the same codebase, and that that same codebase is what is running in the live environment.

By default, Composer packages are drawn from Packagist (<https://packagist.org/>) and there is a repository of WordPress plugins – mirrored from the official wordpress.org repository – at wpackagist.org (<https://wpackagist.org/>). There is also an automatically synced WordPress Core repository in Packagist (<https://packagist.org/packages/johnpbloch/wordpress>) that we will make use of later.

The dependencies are recursive. If a library requires another library as a dependency, Composer will import and manage the second library as well.

Why Git?

Version control systems like Git (<http://git-scm.com>) are a huge help to developers, allowing collaboration on a level that wouldn't be possible without them.

For example, you can use awesome Git features like rolling back breaking changes, merging changes from different developers, and using branches for bug fixes and feature builds. And you can also use amazing tools like GitHub Actions (<https://github.com/features/actions>), Buddy (<https://buddy.works/>), and DeployBot (<https://deploybot.com/>) to automate testing and deployment workflows.

Any files that aren't part of the application shouldn't be included in version control. In WordPress, the most common example of this are media files. You'll want something other than version control if you're trying to keep content in sync, such as WP Migrate (<https://deliciousbrains.com/wp-migrate-db-pro/>), or you could use something like Bill Erickson's "Media from Production" plugin (<https://github.com/billerickson/BE-Media-from-Production>) to avoid having to copy files from your production environment to other environments.

Advantages of the Composer + Git approach

There are pros and cons to using Git and Composer to version control your entire website. It's not suitable for every use case. Using version control is a good way to manage the processes of website changes and updates and to enable collaboration if...

- Your sites are tightly-controlled and firmly your organization's responsibility for the long term.
- You want to enforce a quality assurance process with testing and automated deployment.
- You have a team working together to manage the sites.

The three main advantages to keeping WordPress in Git and using Composer are:

- 1 You get to precisely control the software that is running your website without needing to store all of that software yourself or copy and move it around. You just specify which versions you want, and the automated process can build a copy of your site for staging, for local development, or when deploying to the live environment. This ensures consistency between copies of the site, enabling collaboration and convenience.
- 2 You get version control, ensuring that any changes to the code you are controlling are tracked, attributed, and properly managed. You'll always know exactly which changes were made, who made them, when they were made, and you have the ability to roll back the code to any previously-saved state. And you can use workflows like Pull Requests to conduct code reviews and quality assurance.
- 3 Because the packages are pulled from external sources, you don't need to store all of the external code in your own project's Git repository. You just pull it down from a central source as and when you need it.

Version control gives you the freedom to experiment, while always leaving you with the option of turning back the clock if you don't like the results. This is especially useful in team development situations, when multiple people may be making changes at the same time.


```
$ composer init

Welcome to the Composer config generator

This command will guide you through creating your composer.json config.

Package name (/) [rosswintle/composer-test]:
Description []: A test composer project
Author [Ross Wintle , n to skip]:
Minimum Stability []:
Package Type (e.g. library, project, metapackage, composer-plugin) []: project
License []: GPL

Define your dependencies.

Would you like to define your dependencies (require) interactively [yes]? no
Would you like to define your dev dependencies (require-dev) interactively [yes]? no
Add PSR-4 autoload mapping? Maps namespace "Rosswintle\CTest" to the entered relative path. [src/, n to skip]: n

{
    "name": "rosswintle/composer-test",
    "description": "A test composer project",
    "type": "project",
    "license": "GPL",
    "authors": [
        {
            "name": "Ross Wintle",
            "email": "nom@test.com"
        }
    ],
    "require": {}
}

Do you confirm generation [yes]?
```

There is more configuration to do later, and if you ever get stuck, or just want a starting point, I've created a complete, base `composer.json` (<https://gist.github.com/rosswintle/26dc5a81c83378062a937b49b4d0e58e>) file that you can work from. Feel free to copy and paste or download that to get started.

Installing WordPress

Core Using Composer

We want to install something. So let's start with WordPress core. We saw that it was installable from John Bloch's Composer package (<https://packagist.org/packages/johnpbloch/wordpress>).

The package name for this is just “johnpbloch/wordpress”. You can see this matches the “vendor/name” pattern that we saw before.

So we can add WordPress to our project using the command:

Copy

```
composer require johnpbloch/wordpress
```

This says: “Our project depends on (requires) the johnpbloch/wordpress package. Please add it to the `composer.json` file and install it into our project. With thanks in advance.”

Now, this is a bit of a special package. Normally, Composer installs the dependencies into a subdirectory called `/vendor`. Within the `vendor` directory are subdirectories for each of the vendors and packages.

So you would expect WordPress to now be installed in `/vendor/johnpbloch/wordpress`. But it's not! There is a special script that runs to install WordPress into a `/wordpress` subdirectory of the root of your project. And that is where you should see it now. You will get the `/vendor` directory too, but WordPress has been copied out of it.

If you set up a web server to run with the `/wordpress` directory as a project, you should be able to visit `http://my-domain.local/` (with your domain) and see the WordPress setup screen.

To set it up as a project, you should be able to point MAMP at the `/wordpress` directory, or configure something like Laravel Valet (<https://laravel.com/docs/9.x/valet#custom-valet-drivers>) or run PHP's built-in web server:

Copy

```
php -S localhost:8000 -r wordpress
```

Later we will look at how this directory can be renamed too. In all cases you will need a database running (<https://wordpress.org/support/article/creating-database-for-wordpress/>), which is not covered in this article.

I will leave it up to you to set up the `wp-config.php` file and complete the site's configuration. You may use the setup screen that you now see, the WP-CLI config command (<https://deliciousbrains.com/wpcli-command-structure-guide/#wp-config/>), or you could edit the `wp-config.php` file by hand to get things set up.

The next steps assume you have a site running and being served.

Creating a Git Repository

We have said that one of the main benefits of using Composer in this way is that we can store everything needed to recreate the application code for our site in Git.

There are two steps to beginning this process:

- 1 Initialize a Git repository.
- 2 Add a `.gitignore` file so we don't store files that we can recreate on demand using Composer.

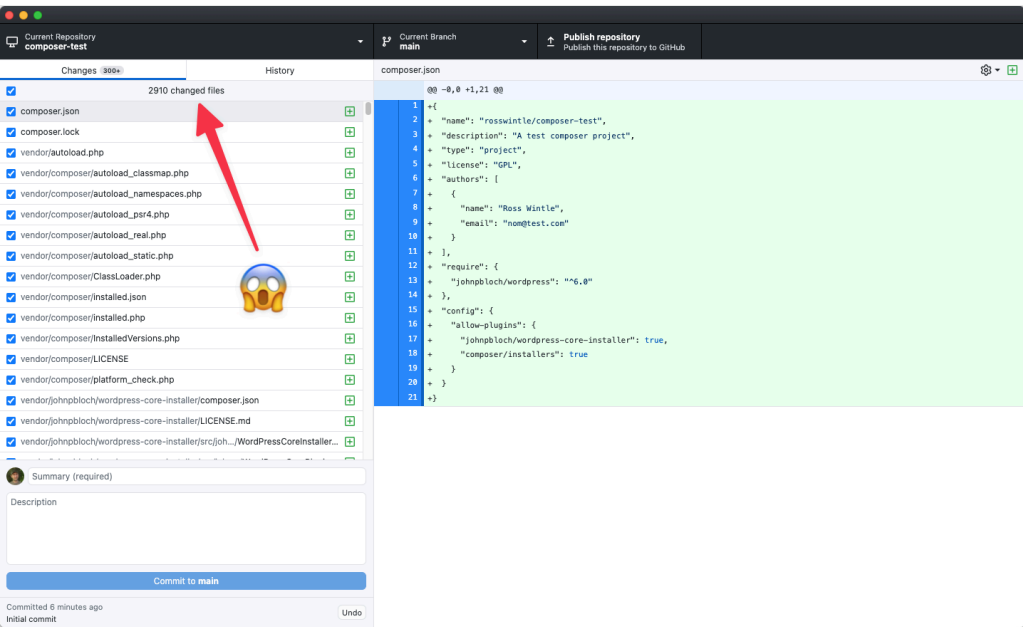
We'll initialize Git in the top-level, root directory of the project. This is *not* the `/wordpress` subdirectory. It is the directory that contains the `composer.json` and `composer.lock` files, and the `/vendor` and `/wordpress` subdirectories.

There are various ways to initialize a Git repository. If you use a GUI client for Git, make sure you “Create from existing files” (or similar) and point it at the project root. Or run `git init` from the command line.

```
$ git init
Initialized empty Git repository in /Users/rosswintle/projects/new-wordpress-site/.git/
```

Copy

If you're using a GUI for Git, you will now see a *lot* of files waiting to be committed.



We don't want this. The aim is not to have WordPress or any of the `/vendor` files in our project repository. We don't need them there, as they can be re-installed at any point using the information in the `composer.lock` file, which defines the current packages and versions in use.

We will use the `.gitignore` file to ... well ... to tell Git to ignore these directories!

Your GUI tool, if you use one, may allow you to do this interactively. Otherwise, use your text editor to create a new file called `.gitignore` in the project root. This should have two lines in it:

```
/vendor
/wordpress
```

Copy

With that saved (or with these directories ignored in your GUI – which, if you're wondering, just creates the `.gitignore` file) you should see just the `.gitignore`, `composer.json`, and `composer.lock` files waiting to be committed.

Use your GUI or command line to commit them (<https://github.com/git-guides/git-commit>), and your project is safely in the hands of Git's clever algorithms.

Adding Plugins and Themes From the WordPress Repository

By default, Composer will only look in the `packagist.org` repository, and it doesn't contain WordPress plugins and themes.

To be able to pull in WordPress plugins and themes, you need to point Composer at the `wpackagist.org` repository. You do this by adding this chunk of config into your `composer.json` file:

```
"repositories": [
{
  "type": "composer",
  "url": "https://packagist.org",
  "only": [
    "wpackagist-plugin/*",
    "wpackagist-theme/*"
  ]
}
],
```

Copy

We also need to tell Composer where to *put* plugins and themes. This involves adding a bit more configuration into `composer.json`:

Copy

```
"extra": {
  "installer-paths": {
    "wordpress/wp-content/mu-plugins/{$name}/": [
      "type:wordpress-muplugin"
    ],
    "wordpress/wp-content/plugins/{$name}/": [
      "type:wordpress-plugin"
    ],
    "wordpress/wp-content/themes/{$name}/": [
      "type:wordpress-theme"
    ]
  }
}
```

You may already have an `extra` section in this file, so be sure to add the new config to it.

With that in place, you can now install any plugin or theme from the official repositories using the `composer require` command, as we did for installing WordPress.

Copy

```
# To install a plugin, use this format:

composer require "wpakagist-plugin/:"

# To install a theme, use this format:
composer require "wpakagist-theme/:"
```

Version constraints (<https://getcomposer.org/doc/articles/versions.md>) are complicated, but probably the simplest form to remember is using a `*` wildcard. To install the free version of our WP Migrate plugin, using version 2.x, you would run:

Copy

```
composer require "wpakagist-plugin/wp-migrate-db:2.*"
```

If you always wanted your updates to get you the latest version, you could use a `*` as the version constraint:

Copy

```
composer require "wpakagist-plugin/wp-migrate-db:*"
```

The first time you run a command like this, you may be asked if you want to “trust “composer/installers” to execute code.” This is safe to do, but be aware that this does let Composer run code on your computer.

If all is well, you should get the plugin installed (but not activated). I would also recommend committing your changes to Git at this point.

Adding Custom or Third-party Plugins and Themes

This is all fine if you want plugins and themes from the repository. But what if you want to add third-party plugins that aren’t on the wordpress.org repository, or your own custom code? The `/wordpress` directory isn’t in Git, so how do you version control your own things?

Some theme and plugin authors support custom repositories for their plugins. For example, we allow users to install WP Migrate via Composer (<https://deliciousbrains.com/wp-migrate-db-pro/doc/installing-via-composer>). But if that doesn’t apply, then the trick here is to selectively unignore specific directories using the `.gitignore` file.

Let’s say you want to create a custom theme with the name `deliciousbrains`. This will be in the `/wordpress/wp-content/themes/deliciousbrains` directory.

Currently, your `.gitignore` file should look like this:

```
/vendor /wordpress
```

We want to ignore this theme directory. We can do that by putting it at the bottom of the file and preceding it with a `!`, but there’s a catch: Unignored files need to be in ignored directories.

For example, unignoring `/wordpress/wp-content/themes/deliciousbrains` doesn’t work because `/wordpress/wp-content/themes` is still ignored. So we need to be a bit cleverer. I can only apologize for this complexity, but feel free to just copy and paste it into your `.gitignore` file:

Exclude composer dependencies in the vendor directory
/vendor

Exclude WordPress core
/wordpress/*
/wordpress/wp-includes/
/wordpress/wp-admin/

Include wp-content, then exclude each of the subdirectories
!/wordpress/wp-content
/wordpress/wp-content/*

Themes: include the directory, exclude all subdirectories except those specified
!/wordpress/wp-content/themes/
/wordpress/wp-content/themes/*
!/wordpress/wp-content/themes/deliciousbrains
Include custom themes here

Plugins: include the directory, exclude all subdirectories except those specified
!/wordpress/wp-content/plugins/
/wordpress/wp-content/plugins/*
Include custom plugins here

Copy

Some Notes on Using Git in This Way

- It’s best practice not to store secure information in a Git repo (you should always ignore `wp-config.php`). This means you’ll have to manually create a `wp-config.php` file on each environment/server that you deploy to.
- Your `/wp-content` folder won’t be stored in Git. If you’re deploying or migrating your site, you’ll need to manually migrate anything that is not installed by composer, such as uploads.
- The database won’t be stored in Git, but don’t worry. WP Migrate (<https://deliciousbrains.com/wp-migrate-db-pro>) has you covered 😊

You’ll also need to commit the changes to your Git repo and re-deploy if you update WordPress locally. Be aware that when using this method WordPress will disable automatic updates (<https://wordpress.org/support/article/configuring-automatic-background-updates/>), because it will detect a `.git` folder. This is a good thing. It means there is a one-way deployment of updates to your site, rather than your live site becoming out of sync with your Git repo. Just make sure you’re aware that keeping your site up-to-date is important for security reasons.

Deployments and Updates

If you deploy your Git repository, you will need to run `composer install` on the server the first time you deploy.

To run updates after that, run `composer update` locally, test to make sure nothing is broken, and commit the `composer.lock` file to Git. Then, when you want to deploy to a live site, run `composer install` on the server after the deployment.

If your `composer.json` file specifies certain versions, such as “2.*”, you may need to update that to reference updated versions of the dependencies too, before running `composer update` . Specifying versions helps you keep critical components (such as WooCommerce) from being updated without manual intervention.

There are also services that will handle most of this for you in one of two ways:

- Running the composer install on a temporary server and then deploying all of the installed files out to your remote server.
- Allowing you to deploy to the server and then automatically running a post-deploy script to install or update the dependencies.

Next Steps:

Moving WordPress

We now have a fully Composer-managed, Git-controlled WordPress install. Hurrah! We are firmly back in control of our software.

We are left, though, with WordPress in the `/wordpress` directory, which may not be ideal.

Some hosting providers let you specify where to serve files from. Others, including your local development environment, may require that they are in a directory named something like `/public` or `/public_html` .

If you want to move your `/wordpress` directory, you can rename it. But you will need to then:

- 1 Add the `wordpress-install-dir` setting to the `extras` section of your `composer.json` .
- 2 Make sure you update paths in both your `.gitignore` file and in your `composer.json` file.
- 3 Run `composer update` and then thoroughly test everything.

After doing such a move, the `extras` section of my `composer.json` looked like this:

```
"extra": {
  "wordpress-install-dir": "public",
  "installer-paths": {
    "public/wp-content/mu-plugins/{%name%}/*": [
      "type:wordpress-muplugin"
    ],
    "public/wp-content/plugins/{%name%}/*": [
      "type:wordpress-plugin"
    ],
    "public/wp-content/themes/{%name%}/*": [
      "type:wordpress-theme"
    ]
  }
}
```

Copy

Next Steps: Using Git Submodules to Manage Themes and Plugins

Using submodules, it's possible to set up a Git repo as a subdirectory of another Git repository, allowing you to clone that repo into your project while keeping your commits separate. There's a good reason to avoid doing this.

It becomes very difficult to make sure the submodules containing your plugins and themes are updated regularly, as you have to do it manually. This will almost certainly slow down your workflow. Even if you've got the time, you'll often find that keeping those submodules updated results in a bloated and unwieldy repo. This can be very tedious to work with, especially if you want to quickly bring new people into your project.

With that said, there are at least two use cases where storing themes and plugins as submodules can really work for you. First, submodules are great if you're developing plugins in WordPress and need to commit your changes back to your repository. You can't do that with Composer. The second case is also applicable to plugin development. If you develop a plugin yourself, one that you maintain and reuse across multiple sites, then storing your plugin in a Git submodule can end up saving you a lot of time.

Next Steps: Restructuring WordPress

There are some benefits to having WordPress core files separate from the other parts of your site. Iain covered some of the benefits in his post when he detailed how to install WordPress core in a subdirectory using WP-CLI or Composer (<https://deliciousbrains.com/move-wordpress-root-subdirectory/#composer>).

This is a more advanced topic, so I'm not going to cover it here, but that article is a good first step.

Wrapping Up

It might seem like we've done a lot of work but this is where we begin to see the advantage of a setup like this. Now we just need to run `composer install` to install WordPress or `composer update` to update WordPress.

All of our team can very easily get the exact, current state of our site's code. We are storing minimal code in our Git repository, instead relying on defined dependencies to state what should be installed, installing those dependencies on demand, and large deployments can be simplified and easily automated.

I hope you're able to try this out, see the benefits, and take some, or all, of this workflow and implement it in your own WordPress sites.

Remember that this is not necessarily the "right" way to manage WordPress. It is just one way you can use WordPress, Git and Composer all together. Take the bits that work for you and create your own workflow and ignore the rest, or improve this workflow. It's up to you. Got any tips or suggestions we didn't cover? Let us know in the comments.

This entry was posted in [WP Migrate DB Pro \(https://deliciousbrains.com/wp-migrate-db-pro/level-up/\)](https://deliciousbrains.com/wp-migrate-db-pro/level-up/), [Workflow \(https://deliciousbrains.com/wp-migrate-db-pro/level-up/workflow/\)](https://deliciousbrains.com/wp-migrate-db-pro/level-up/workflow/) and tagged [WordPress \(https://deliciousbrains.com/tag/wordpress/\)](https://deliciousbrains.com/tag/wordpress/), [Development \(https://deliciousbrains.com/tag/development/\)](https://deliciousbrains.com/tag/development/), [Deployment \(https://deliciousbrains.com/tag/deployment/\)](https://deliciousbrains.com/tag/deployment/), [CLI \(https://deliciousbrains.com/tag/cli/\)](https://deliciousbrains.com/tag/cli/), [Composer \(https://deliciousbrains.com/tag/composer/\)](https://deliciousbrains.com/tag/composer/), [Git \(https://deliciousbrains.com/tag/git/\)](https://deliciousbrains.com/tag/git/).

ABOUT THE AUTHOR



Ross Wintle, Software Developer

Ross is a software developer based in Swindon in the UK who specialises in WordPress and Laravel. He enjoys solving all kinds of complex, real world problems with code and loves helping other developers out. In a previous life he worked on aerospace systems; websites have always felt more down to earth!

@magicroundabout (<https://x.com/magicroundabout>)

rosswintle.uk (<https://rosswintle.uk/>)

that unfortunately.

Reply ^ Upvote

SS

ssteinerX

...

I've now refined it so that we only store the composer.json file, and a couple of control files (.htaccess etc.) in Git with composer handling all dependencies (including WordPress!) either from wpackagist (which mirrors all WordPress.org components) or a self-hosted Composer repo. Gets around the paid plugin issue.

Reply ^ Upvote

ND

Ned Daze

...

Really interesting post. I've been looking at jumping into version control for the WP install at my day job. Having 3-4 different contractors working on the site at any given time makes troubleshooting a bit of a headache. I was looking at WP Pusher to just keep themes and custom plugins in Git repositories, but had never thought of applying these principles to the whole site. Looking forward to the upcoming posts on how you make this work.

Reply ^ Upvote

AA

Ahmad Awais

...

Hey Gilbert nice set of series. But one question, why would anyone want to store WP in Git? I mean if you developing theme or a plugin, just using Git for them is enough. Why upload WordPress to Git?

Reply ^ Upvote

AA

Ahmad Awais

...

Yup that makes sense.

Reply ^ Upvote

SS

ssteinerX

...

We just use Scriptaculous to create a raw install, import a database backup (using WP Migrate Pro), and check the wp-content directory out of git. I haven't read the rest of these posts, but checking WordPress into a git repo just seems wrong unless you're mirroring the WordPress repo and selecting a particular revision from a central repo which might make some sense in some cases.

Reply ^ Upvote

JJ

Jake Jackson

...

If you want a full-blown version-controlled composer stack with deployment I'd strongly recommend Bedrock, by roots (<https://roots.io/bedrock/>). It will save you hours trying to set this up and we use it daily to easily make changes and deploy / rollback changes.

Reply ^ Upvote

SS

ssteinerX

...

Every time I look at Bedrock, I marvel at how complicated they've made a fairly straightforward process.

Reply ^ Upvote

JN

Jonathan Nicol

...

The approach advocated by this article seems counter intuitive. The WordPress core is a third party dependency, so I'm not sure what benefit there is to keeping it under version control? In my workflow I use Git to store custom code, which in the case of a WordPress site is the site's theme and any site-specific plugins, but the article recommends excluding the wp-content directory from Git. What is your rationale for keeping your custom code outside of Git?

Reply ^ Upvote

AA

Ahmad Awais

...

Yup that makes sense.

Reply ^ Upvote

SS

ssteinerX

...

If you don't store uploads, how does the media library get filled?

Reply ^ Upvote

TM

Travis Miller

...

It doesn't. Media files are content, just like posts. They aren't part of the application, and therefore shouldn't be included in version control. If you're trying to keep content in sync (e.g., between dev and production instances of a site), then you probably want a tool other than version control.

Reply ^ Upvote

BS

Bharat Singhal

...

Thanks for the nice article, but one question I have is that if I install wordpress on a PaaS server then installing plugins on the live website through GUI will not be persistent if the server is rebooted, so how to manage wordpress as a whole with git?

Reply ^ Upvote

PW

Patrick W

...

I've used WP Migrate DB Pro for about 2 years ever since you sponsored a ShoptalkShow podcast and it totally changed my dev workflow. Now I'm finally getting onto the GIT and WP bandwagon and this post is excellent. I've also used WP Offload S3 recently and I have to say that your plugins are amongst the best I've used. Thanks so much for writing this tutorial.

Reply ^ Upvote

CL

Connor Leech

...

So how would you go using this git setup to work with many clients (100+ WP installation) ? You would have 1 repo per client? And then, what about using plugin like MainWP to mass update themes/plugins and stuff like that? Is that possible with such a mass of working? I'm thinking of using Vagrant as well now. idea?

Reply ^ Upvote

SG

Srikanth Ganta

...

Good article

Reply ^ Upvote

RO

rogerdpack

...

i don't get it when you ignored all in wp-content to git i think we should ignored the core, because it's generated file, we can download it again with WP CLI in deployment if you wanna ignore the wp-content, you can ignored the "upload" because i think you can use cloud storage instead of that and ignored the plugin

Reply ^ Upvote

SIGN UP FOR THE LATEST PRODUCT NEWS AND UPDATES

we want the vendor directory in our .gitignore? Also, is there source code for this project available on github? Thank you for the tutorial

SUBSCRIBE

Products

ACF

Advanced Custom Fields(<https://advancedcustomfields.com/>)

(https://x.com/wp_acf)

Home (<https://deliciousbrains.com/>)

...

WP Migrate

WP Migrate(<https://deliciousbrains.com/wp-migrate-db-pro/>)

(<https://x.com/wpmigratedbpro>)

Blog (<https://deliciousbrains.com/blog/>)

...

WP Offload Media

WP Offload Media(<https://deliciousbrains.com/wp-offload-media/>)

(<https://x.com/wpoffloadmedia>)

Podcast (<https://podcast.deliciousbrains.com/>)

...

WP Offload SES

WP Offload SES(<https://deliciousbrains.com/wp-offload-ses/>)

(<https://x.com/wpoffloads>)

Contact Us (<https://deliciousbrains.com/contact-us/>)

...

Company

My Account (<https://deliciousbrains.com/my-account/>)

...

About (<https://deliciousbrains.com/about/>)

can ignore the upload because I think you can use cloud storage instead of that and ignore the plugin, because we use composer for that right ?

Reply ^ Upvote

Careers (<https://wpengine.careers/>)

Vasily Artamonov

Press (<https://deliciousbrains.com/press/>)

The goals the author of this article tries to accomplish are absolutely the right way to go with WordPress development.

(<https://deliciousbrains.com/>)

However, to not reinvent the wheel yourself I suggest trying out Bedrock by Roots:

<https://roots.io/bedrock/>. This boilerplate already does everything out of the box:

Delicious Brains Inc.

(<https://deliciousbrains.com/>)

Composer folder structure;

- gitignore file;

@dliciousbrains

(<https://x.com/dliciousbrains>)

Composer project config, including setup for plugin and themes installation from WPackagist;

- separating configuration for different environments;
- moving WordPress core into it's own directory;
- and more.

© 2013–2025 WPENGINE, INC. ALL RIGHTS RESERVED.

I highly recommend checking out Bedrock. And, to go further, the whole Bedrock ecosystem of open-source plugins and themes.

PRIVACY POLICY
(<https://wpengine.com/legal/privacy/>)

TERMS AND CONDITIONS
(<https://wpengine.com/legal/terms-of-service/>)

WP ENGINE IS A PROUD MEMBER AND SUPPORTER OF THE COMMUNITY OF WORDPRESS® USERS. THE WORDPRESS® TRADEMARKS ARE THE INTELLECTUAL PROPERTY OF THE WORDPRESS FOUNDATION, AND THE WOO® AND WOOCOMMERCE® TRADEMARKS ARE THE INTELLECTUAL PROPERTY OF WOOCOMMERCE, INC. USES OF THE WORDPRESS®, WOO®, AND WOOCOMMERCE® NAMES IN THIS WEBSITE ARE FOR IDENTIFICATION PURPOSES ONLY AND DO NOT IMPLY AN ENDORSEMENT BY WORDPRESS FOUNDATION OR WOOCOMMERCE, INC. WP ENGINE IS NOT ENDORSED, OWNED, BY, OR AFFILIATED WITH, THE WORDPRESS FOUNDATION OR WOOCOMMERCE, INC.



Composer is nice in theory, but for a team of developers it quickly gets unreliable. It often fails and then whoever wrote the composer setup has to look at it to see what the problem is. It's more reliable and straight-forward to just include the required files with the themes and plugins, and then push the updates, instead of hoping that everyone's composer setup is identical enough to always work.

Reply ^ Upvote



Simon Frost